Custom App Development Project Setup

https://gemini.google.com/app/9a5e91cccf1da945

User prompt: Https://www.linkedin.com/jobs/view/4463306629/ Research this company, what the company currently building and make it a project for me to do, requirements include me learning how to do build an app,database, cicd workflow, ai agent assisted coding workflow, version control, debugging(ai assisted), automated testing, cicd

Response: LinkedIn blocks external crawlers and unauthenticated access to specific `/jobs/view/` URLs, and job ID `4463306629` isn't indexed on the public web yet.

Drop the **company name and job title** (or paste the raw text of the job description), and I will immediately research their current product roadmap and build out a custom end-to-end project tailored to their stack—covering the app architecture, database schema, Git version control, AI agent-assisted coding/debugging, automated testing, and a full CI/CD pipeline.

User prompt: About the job About Ant International Ant International is a leading global digital payment, digitisation and financial technology provider. Through collaboration across the private and public sectors, our unified techfin platform supports financial institutions and merchants of all sizes to achieve inclusive growth through a comprehensive range of cutting-edge digital payment and financial services solutions. Ant International's Global Development Centre (GDC) is located at The Exchange 106, within TRX, Malaysia's dedicated international financial centre. The GDC is Ant International's strategic hub for world-class talent, driving the innovations that advance inclusive global commerce. It represents another milestone in Ant International's journey in Malaysia, reflecting the company's belief that world-class innovation can be developed in Malaysia for global markets, by creating high-value employment opportunities, nurturing local talent and collaborating with public and private sector partners. About the Role Our Malaysia Tech Center is advancing cross-border payments, SME digitalisation, and next-generation financial infrastructure, supporting global businesses and products such as WorldFirst, Alipay+, Antom, and Bettr. As an Applied AI Engineer, you will own end-to-end product engineering delivery in an AI-native environment. AI is deeply embedded in how we clarify requirements, design solutions, build software, alongside how we test, deploy, and continuously improve our products—it is not merely an occasional coding aid. You will collaborate with AI agents throughout the delivery lifecycle while applying sound engineering judgment, retaining full accountability for the correctness, security, reliability, and business outcomes of everything delivered. Your Mission Contribute to clearly scoped product features and fixes under guidance, from requirement clarification to deployment. Build and maintain backend services and APIs following established engineering standards. Ensure code quality through rigorous validation, proactive defect resolution, and response to production feedback. Work within established AI-native engineering workflows, collaborating with AI agents across implementation, testing, debugging, and documentation while validating every output. Participate in technical topic, code reviews, communicate delivery progress or risks timely, and contribute to team knowledge sharing. Minimum Qualifications Currently pursuing or recently completed a Bachelor's Degree in Computer Science, Software Engineering, or a related field, or have equivalent practical experience. Foundational programming proficiency in at least one general-purpose language. Conceptual understanding of APIs, databases, version control, and core software development concepts. Proven ability to build and debug working software through coursework, internships, or projects, with strong learning, ownership, and communication skills. Preferred Qualifications Internship or project experience building a backend service or API connected to a database. Familiarity with automated testing and software quality practices. Experience using AI-native development workflows, including providing context to AI agents and reviewing the outputs.

Response: 1. Company Breakdown: What Ant International (GDC Malaysia) is Building

Ant International (spun off as an independent operating unit from Ant Group, headquartered in Singapore with its flagship Global Development Centre at **The Exchange 106, TRX, Kuala Lumpur**) builds the foundational rails for cross-border digital commerce. Their core ecosystem consists of four pillars mentioned directly in the JD, plus two major AI/FinTech initiatives:

- **Alipay+:** A cross-border mobile payment and digitalization technology suite that connects global merchants with e-wallets (like Touch 'n Go eWallet, GCash, KakaoPay, Dana) so users pay in their home currency while merchants receive settlements in theirs.
- **Antom:** Global merchant payment and acquiring services—handling credit cards, local payment methods, intelligent payment routing, and checkout risk/fraud control for enterprise merchants.
- **WorldFirst:** Cross-border B2B payment and treasury platform for SMEs and e-commerce sellers, offering multi-currency accounts, foreign exchange (FX) hedging, and supplier payouts.
- **Bettr:** Digital lending and embedded micro-financing for SMEs in emerging markets, relying on alternative data credit scoring and automated risk engines.
- **Falcon & AI-Native Engineering:** Internally and externally, Ant International invests heavily in **Falcon** (their Time-Series Transformer AI model used to forecast FX liquidity and reduce currency conversion volatility) and **AI-Native SDLC** (using agentic coding tools, Model Context Protocol [MCP] integrations, and automated code review/testing pipelines).

What this specific role ("Applied AI Engineer") actually is: Despite the title, this is **not** a pure model-training research role. It is an **AI-Native Backend/Full-Stack Software Engineering role** where you build mission-critical payment/FinTech microservices (APIs, databases, transaction states) by orchestrating AI coding agents across the entire Software Development Life Cycle (SDLC)—from writing specs and generating code to automated testing, security validation, and CI/CD deployment.

2. Your Capstone Project: "AntFlow Guard"

Project Title: Cross-Border Payment Routing, FX Settlement & AI Anomaly Triage Engine

The Pitch: Build a mini-clone of the **Antom / WorldFirst** backend—a high-reliability cross-border merchant settlement API that processes multi-currency transactions, guarantees zero double-charging via idempotency, evaluates transaction risk/FX anomalies in real-time, and is built, tested, debugged, and deployed using a strict **AI-Native Engineering Workflow**.

Tech Stack

- **Backend API:** Python (**FastAPI** + Pydantic v2) — clean async support, strict type validation, auto-generated OpenAPI docs.
- **Frontend Dashboard: Streamlit** (or Next.js) — live operator console showing transaction feeds, FX spread savings, and flagged risk alerts.

- **Database: PostgreSQL** (run via Docker) + **SQLAlchemy 2.0 (Async)** + **Alembic** (database schema migrations).
- **AI Agent & MCP Workflow:** Google Antigravity / Claude Code / Cursor connected to a local **PostgreSQL MCP server** and **GitHub MCP server**, governed by strict context rules.
- **Testing & Quality:** `pytest`, `httpx` (async API testing), `hypothesis` (property-based testing for financial edge cases), `ruff` (linter), `bandit` (SAST security scanner).
- **CI/CD & DevOps: Docker Compose** + **GitHub Actions** (automated lint, security scan, Spin-up Postgres service container, run migrations, execute test suite, and auto-triage failures).

3. Core Architecture & Database Design

In FinTech interviews at Ant International, the biggest differentiator is knowing how to handle **money safely in a database**. Never use floating-point numbers (`float`) for currency, and always enforce **Idempotency** (so a user clicking "Pay" twice on a laggy connection isn't charged twice).

Database Schema (PostgreSQL)

Table Name	Key Columns	Engineering Purpose (Why Ant Cares)
`merchants`	`id (UUID)`, `name`, `home_currency (MYR, SGD, USD)`, `status`, `created_at`	Represents an Antom merchant or WorldFirst SME account.
`wallets`	`id (UUID)`, `merchant_id (FK)`, `currency`, `balance (NUMERIC(18,4))`, `version (INT)`	Uses `NUMERIC(18,4)` to prevent floating-point rounding errors. Includes `version` for **Optimistic Locking** to prevent race conditions during concurrent balance updates.
`fx_rates`	`id`, `base_currency`, `target_currency`, `rate (NUMERIC(18,6))`, `volatility_score`, `timestamp`	Simulates the **Falcon FX Engine** feed for cross-border conversion (e.g., CNY $\rightarrow$ MYR).
`transactions`	`id (UUID)`, `idempotency_key (UNIQUE)`, `merchant_id`, `source_amount`, `source_currency`, `settled_amount`, `settled_currency`, `fx_rate_applied`, `status (PENDING, SETTLED, FLAGGED, REJECTED)`, `latency_ms`	The core ledger. The `UNIQUE` constraint on `idempotency_key` guarantees exactly-once processing.
`risk_audit_logs`	`id (UUID)`, `transaction_id (FK)`, `risk_score (0–100)`, `anomaly_reason (JSONB)`, `reviewed_by_agent (BOOL)`	Stores explainable risk triage results (e.g., velocity spikes, abnormal FX spread slippage, or anomalous payload latency).

4. Step-by-Step Implementation Blueprint (Meeting All Your Requirements)

Phase 1: AI-Agent Assisted Coding Workflow (Context Engineering)

The JD explicitly highlights: *"providing context to AI agents and reviewing the outputs."* Don't just chat with an LLM; set up a formal **Spec-Driven Development (SDLC)** structure in your repo so you can show interviewers *how* you govern AI agents.

1. **Create an Agent Rules File (`.cursorrules`, `CLAUDE.md`, or `.antigravity/rules.md`):** Commit this file directly into your repository so hiring managers can see it:

   ```
   # AI Engineering Constitution for AntFlow-Guard
   1. FINANCIAL PRECISION: Never use `float` for monetary amounts or FX rates. Strictly use Python `decim
   2. CONCURRENCY & IDEMPOTENCY: Every `POST /v1/payments` endpoint MUST require an `X-Idempotency-Key` h
   3. SECURITY: Never log raw PII or API keys. Validate all inputs via Pydantic v2 strict mode.
   4. TDD MANDATE: For every new endpoint or service function generated, you MUST generate corresponding
   ```

2. **Spec-First Prompting (`docs/specs/`):** Before writing code for a feature, write a 1-page Markdown spec in `docs/specs/01-payment-settlement.md` defining the input JSON, state machine transitions (`PENDING` $\rightarrow$ `SETTLED`), and error codes. Instruct your AI agent to read the spec and generate the SQLAlchemy models, Alembic migration, and FastAPI router.

Phase 2: Building the App & Database Layer

1. **Spin up Local Infrastructure (`docker-compose.yml`):** Configure two services: `db` (`postgres:16-alpine`) and `api` (your FastAPI app).
2. **Build the Core API Endpoints:**
   - `POST /api/v1/payments/settle`: Accepts a cross-border payment request (e.g., Customer pays 1,000 CNY via Alipay+, Merchant in KL receives MYR).
     - Checks `idempotency_key`. If it already exists, immediately return the cached response (`200 OK`) without touching the database balance.
     - Fetches the latest FX rate and computes settled amount using `Decimal` with `ROUND_HALF_EVEN` (banker's rounding).
     - Runs the **Risk & Anomaly Check**: flags transactions where the amount exceeds 3σ of the merchant's historical rolling average or where system execution latency spikes anomalously.

        - Executes an atomic database transaction (`async with session.begin():`) to credit the merchant wallet and insert the transaction record.
    - `GET /api/v1/merchants/{id}/reconciliation`: Generates a daily settlement summary comparing expected balances vs. ledger entries.
3. **Build the Operator Dashboard (`dashboard.py`):** A quick Streamlit app calling your FastAPI endpoints to visualize live cross-border payment flows, currency corridor volumes (MYR/SGD/CNY/USD), and flagged transactions requiring review.

Phase 3: Version Control (Git) & Engineering Standards

Treat this solo project like a production enterprise repository at TRX:

1. **Branching Strategy:** Never commit directly to `main`. Use feature branches:
    - `feat/db-schema-and-migrations`
    - `feat/idempotent-payment-api`
    - `fix/race-condition-wallet-lock`
    - `ci/github-actions-pipeline`
2. **Conventional Commits:** Enforce clean commit messages (`feat(payment): add row-level locking on wallet balance mutation`).
3. **Pull Request Templates & AI Self-Review:** Create `.github/pull_request_template.md` with sections for *Architecture Changes*, *Database Migration Impact*, *Security/Financial Edge Cases*, and *AI Agent Verification Notes* (documenting where the AI initially made a mistake—like forgetting a rollback—and how your human engineering judgment caught and fixed it).

Phase 4: Automated Testing (Unit, Concurrency & Property-Based)

FinTech interviewers love candidates who test beyond basic `assert 200 == 200`. Build a 3-tier test suite in `tests/`:

1. **Unit Tests (`tests/unit/test_fx_calculator.py`):** Use **Hypothesis** (property-based testing) to throw 500 randomized currency amounts and FX rates at your conversion engine, verifying that `settled_amount` never loses sub-cent precision or produces NaN/negative numbers.
2. **Integration & Idempotency Tests (`tests/integration/test_payments_api.py`):** Fire the exact same `POST /api/v1/payments/settle` payload with `X-Idempotency-Key: tx-999` 5 times in a row. Assert that the merchant's wallet balance only increases **once**.
3. **Concurrency / Race Condition Test (`tests/integration/test_concurrency.py`):** Use `asyncio.gather()` to fire 20 simultaneous payout requests against a merchant wallet that only has enough balance for 5 payouts. Assert that exactly 5 succeed (`200 OK`), 15 fail cleanly with `400 Insufficient Funds`, and the final wallet balance is never negative.

Phase 5: AI-Assisted Debugging Drill (Documented Case Study)

The JD asks for: *"collaborating with AI agents across implementation, testing, debugging, and documentation while validating every output."* Create a dedicated folder `docs/postmortems/` and intentionally simulate, debug, and document two classic production bugs using your AI agent:

- **Bug Drill 1: The Double-Spend Race Condition:**
    - *Setup:* Write the initial wallet update without `with_for_update()` (row locking). Run your `asyncio.gather()` concurrency test and watch it fail (allowing the wallet balance to drop below zero or miscalculate).
    - *AI Debugging Workflow:* Pipe the `pytest` stack trace and SQL query logs into your AI agent via terminal/MCP. Prompt it to analyze the transaction isolation anomaly (Read Committed vs. Repeatable Read / Row Locking).
    - *Resolution:* Implement `select(Wallet).where(...).with_for_update()`, re-run the concurrency test to prove it passes, and save the root-cause analysis in `docs/postmortems/001-wallet-race-condition.md`.
- **Bug Drill 2: Silent Performance/Latency Degradation:**
    - *Setup:* Introduce an $N + 1$ query bottleneck in the `GET /reconciliation` endpoint when fetching transaction audit logs.
    - *AI Debugging Workflow:* Capture the endpoint execution time and SQLAlchemy `echo=True` query logs, feed them to the agent to diagnose the $N + 1$ issue, and refactor to `selectinload()`.

Phase 6: CI/CD Workflow (GitHub Actions + Security + AI Triage)

Create `.github/workflows/ci-cd.yml` so every `git push` and Pull Request triggers an automated enterprise pipeline:

```
name: AntFlow Guard CI/CD Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

```
jobs:
  quality-and-security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install Linters & Security Scanners
        run: pip install ruff bandit mypy
      - name: Run Ruff (Lint & Format Check)
        run: ruff check .
      - name: Run Bandit (SAST Security Scan for FinTech vulnerabilities)
        run: bandit -r app/ -ll

  integration-tests:
    needs: quality-and-security
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: ant_test
          POSTGRES_PASSWORD: ant_test_password
          POSTGRES_DB: antflow_test_db
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-timeout 5s
          --health-retries 5
    env:
      DATABASE_URL: postgresql+asyncpg://ant_test:ant_test_password@localhost:5432/antflow_test_db
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install Dependencies
        run: pip install -r requirements.txt
      - name: Run Alembic Database Migrations
        run: alembic upgrade head
      - name: Run Pytest (Unit, Idempotency & Concurrency Suite)
        run: pytest --cov=app --cov-report=xml -v
```

Bonus CI/CD Flex (AI Failure Triage Step): Add a conditional step (`if: failure()`) at the end of your GitHub Actions workflow that takes the failed `pytest` log output, sends it to an LLM API (Gemini Flash / OpenAI), and automatically posts a comment on the Pull Request explaining the root cause of the test failure and suggesting a patch diff.

5. How to Pitch This on Your Resume & in the Ant International Interview

Once built, add this to your CV and GitHub README using the exact terminology from the job description:

- **Project Header: AntFlow Guard — AI-Native Cross-Border Payment & FX Settlement Engine** | *Python, FastAPI, PostgreSQL, SQLAlchemy, Docker, GitHub Actions*
- **Resume Bullet 1 (Core Backend & DB):** Engineered an asynchronous cross-border payment settlement API and multi-currency ledger in **FastAPI** and **PostgreSQL**, enforcing strict `NUMERIC(18,4)` precision, `X-Idempotency-Key` deduplication, and pessimistic row-level locking (`SELECT FOR UPDATE`) to eliminate double-spend race conditions under concurrent load.
- **Resume Bullet 2 (AI-Native SDLC):** Designed a spec-driven, **AI-native engineering workflow** utilizing context constitution rules (`CLAUDE.md`) and MCP database inspection to accelerate boilerplate generation, schema migrations, and root-cause debugging while retaining strict human verification over financial edge cases.
- **Resume Bullet 3 (Testing & CI/CD):** Built a multi-stage **GitHub Actions CI/CD pipeline** integrating **Bandit SAST** security scanning, ephemeral PostgreSQL service containers, Alembic migrations, and automated `pytest` + `hypothesis` property-based test suites with automated AI log triage on build failures.

Would you like to kick off **Phase 1 & 2** right now by generating the project directory structure, the `CLAUDE.md` / `.cursorrules` AI constitution file, and the core PostgreSQL + FastAPI code for the idempotent payment settlement endpoint?

User prompt: yes proceed. also how do i make sure i learn from this when i do this in antigravity

Response: To genuinely learn while using an agentic IDE like **Google Antigravity**—rather than passively clicking "Accept All" and ending up with a codebase you can't explain in an interview—you need to treat the AI as a **Junior Pair Programmer** while you act as the **Staff Engineer /**

Architect.

Part 1: The "Anti-Autopilot" Protocol (How to Actually Learn in Antigravity)

Ant International's JD specifically states: *"collaborate with AI agents throughout the delivery lifecycle while applying sound engineering judgment, retaining full accountability..."* Here is the exact workflow to build deep muscle memory:

1. **The "Spec-First, Code-Second" Rule:** Never ask Antigravity to "build the payment app." Write the data types, state transitions, and failure modes yourself in a Markdown spec first. If you can't specify it, you don't understand it.
2. **Enforce "Socratic Diff Review":** Configure Antigravity's system rules so that before it modifies any file, it must explain **why** it chose a specific SQL query or concurrency pattern and ask you a verification question.
3. **Write the Critical Tests Yourself (or Break Them Intentionally):** Have the agent scaffold the boilerplate, but **you** write the assertions in `test_concurrency.py` or intentionally comment out `with_for_update()` to watch the database corrupt under concurrent load. Seeing a test fail and reading the PostgreSQL deadlock/race log teaches you more than 10 hours of tutorials.
4. **Keep an `AI_ENGINEERING_LOG.md`:** Every time Antigravity suggests suboptimal code (e.g., using `float` instead of `Decimal`, forgetting an index on `idempotency_key`, or using deprecated Pydantic v1 syntax), log it in this file. During your interview at TRX, pulling up this log proves you actually govern the AI rather than blindly trusting it.

Part 2: Project Scaffolding & Git Initialization

Open your terminal on macOS, create the repository, and set up your first feature branch so you practice clean version control from minute one:

```
mkdir antflow-guard && cd antflow-guard
git init -b main

# Create the enterprise folder structure
mkdir -p .antigravity docs/specs docs/postmortems app/api app/core app/models app/schemas app/services tes

# Create initial files
touch .antigravity/rules.md AI_ENGINEERING_LOG.md docker-compose.yml requirements.txt
touch app/__init__.py app/main.py app/core/database.py app/models/ledger.py app/schemas/payment.py app/ser

# Create your first feature branch (Never commit directly to main!)
git checkout -b feat/core-ledger-and-idempotency
```

Part 3: Your Antigravity Constitution (`.antigravity/rules.md`)

Paste this into `.antigravity/rules.md` (and/or `CLAUDE.md` in the root). Notice **Rule 5**—it forces Antigravity into "Learning/Mentor Mode" so you never passively copy-paste:

```
# AntFlow Guard — AI Engineering & Learning Constitution

## 1. Domain Context
We are building AntFlow Guard, a high-reliability cross-border payment settlement and FX risk microservice

## 2. Non-Negotiable Financial Engineering Standards
- **NO FLOATS FOR MONEY:** Strictly use Python `decimal.Decimal` and PostgreSQL `NUMERIC(18, 4)` for monet
- **IDEMPOTENCY:** All state-mutating endpoints (`POST /api/v1/payments/settle`) MUST require an `X-Idempo
- **CONCURRENCY CONTROL:** Any database query reading a `Wallet` to mutate its `balance` MUST use pessimis
- **MODERN STACK:** Use SQLAlchemy 2.0 `Mapped` / `mapped_column` syntax (`AsyncSession`), Pydantic v2 (`m

## 3. Security & Observability
- Never log raw credentials or PII.
- Calculate and store execution latency and statistical Z-score anomaly flags in `risk_audit_logs` for eve

## 4. Testing Mandate
- Every service method requires both unit tests (`pytest` + `hypothesis` for boundary math) and async inte

## 5. ACTIVE LEARNING MODE (CRITICAL)
- Do NOT dump massive multi-file rewrites without explanation.
- For every architectural change (DB locking, async sessions, CI/CD config), briefly explain the underlyin
- After generating a complex module, challenge the human engineer with 1 pointed edge-case question to ver
```

Part 4: Phase 1 & 2 Core Implementation

Rather than letting the agent generate everything at once, review and implement these foundational files step-by-step.

1. Local Infrastructure (`docker-compose.yml` & `requirements.txt`)

`requirements.txt`

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
sqlalchemy[asyncio]==2.0.35
asyncpg==0.29.0
alembic==1.13.2
```

```
pydantic==2.9.2
httpx==0.27.2
pytest==8.3.3
pytest-asyncio==0.24.0
hypothesis==6.112.1
ruff==0.6.8
bandit==1.7.10
streamlit==1.38.0
```

docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    container_name: antflow_postgres
    environment:
      POSTGRES_USER: ant_dev
      POSTGRES_PASSWORD: ant_dev_password
      POSTGRES_DB: antflow_db
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ant_dev -d antflow_db"]
      interval: 5s
      timeout: 5s
      retries: 5
```

2. Async Database Engine (`app/core/database.py`)

Here you learn how **SQLAlchemy 2.0 AsyncEngine** manages connection pooling for high-throughput microservices:

```
import os
from typing import AsyncGenerator
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine
from sqlalchemy.orm import DeclarativeBase

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql+asyncpg://ant_dev:ant_dev_password@localhost:5432/antflow_db",
)

engine = create_async_engine(
    DATABASE_URL,
    echo=False,  # Set to True during debugging drills to inspect raw SQL!
    pool_size=20,
    max_overflow=10,
)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
)


class Base(DeclarativeBase):
    pass


async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        yield session
```

3. Financial Ledger Models (`app/models/ledger.py`)

Study how `Numeric` types and `UniqueConstraint` protect data integrity at the database level—even if application code has a bug:

```
import uuid
from datetime import datetime, timezone
from decimal import Decimal
from sqlalchemy import DateTime, ForeignKey, Numeric, String, JSON
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column
from app.core.database import Base


class Wallet(Base):
    __tablename__ = "wallets"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    merchant_id: Mapped[str] = mapped_column(String(64), index=True, nullable=False)
    currency: Mapped[str] = mapped_column(String(3), nullable=False)  # e.g., "MYR", "SGD"
    balance: Mapped[Decimal] = mapped_column(Numeric(18, 4), nullable=False, default=Decimal("0.0000"))
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=lambda: datetime.now(timezone.utc)
    )


class Transaction(Base):
    __tablename__ = "transactions"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    idempotency_key: Mapped[str] = mapped_column(String(128), unique=True, index=True, nullable=False)
    merchant_id: Mapped[str] = mapped_column(String(64), index=True, nullable=False)
    source_amount: Mapped[Decimal] = mapped_column(Numeric(18, 4), nullable=False)
    source_currency: Mapped[str] = mapped_column(String(3), nullable=False)
    fx_rate_applied: Mapped[Decimal] = mapped_column(Numeric(18, 6), nullable=False)
    settled_amount: Mapped[Decimal] = mapped_column(Numeric(18, 4), nullable=False)
    settled_currency: Mapped[str] = mapped_column(String(3), nullable=False)
    status: Mapped[str] = mapped_column(String(20), nullable=False)  # SETTLED, FLAGGED_HIGH_RISK
    risk_score: Mapped[int] = mapped_column(nullable=False, default=0)
    risk_metadata: Mapped[dict] = mapped_column(JSON, default=dict)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=lambda: datetime.now(timezone.utc)
    )
```

4. Strict Pydantic v2 Schemas (`app/schemas/payment.py`)

```
import uuid
from decimal import Decimal
from pydantic import BaseModel, ConfigDict, Field


class PaymentSettlementRequest(BaseModel):
    merchant_id: str = Field(..., min_length=3, max_length=64, examples=["merch_trx_kl_01"])
    source_amount: Decimal = Field(..., gt=Decimal("0.00"), decimal_places=4, examples=[Decimal("1000.0000
    source_currency: str = Field(..., pattern="^[A-Z]{3}$", examples=["CNY"])
    settled_currency: str = Field(..., pattern="^[A-Z]{3}$", examples=["MYR"])
    fx_rate: Decimal = Field(..., gt=Decimal("0.00"), examples=[Decimal("0.648520")])


class PaymentSettlementResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    idempotency_key: str
    merchant_id: str
    source_amount: Decimal
    source_currency: str
    settled_amount: Decimal
    settled_currency: str
    fx_rate_applied: Decimal
    status: str
    risk_score: int
     new_wallet_balance: Decimal
    idempotent_replay: bool = False
```

5. The Core Settlement & Risk Engine (`app/services/settlement.py`)

Read every line of `settle_cross_border_payment` carefully—this single function demonstrates **Banker's Rounding**, **Idempotency Deduplication**, **Pessimistic Row Locking (`with_for_update`)**, and **Automated Risk Triage**:

```
from decimal import Decimal, ROUND_HALF_EVEN
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from app.models.ledger import Transaction, Wallet
from app.schemas.payment import PaymentSettlementRequest, PaymentSettlementResponse

FOUR_PLACES = Decimal("0.0001")


def calculate_settled_amount(source_amount: Decimal, fx_rate: Decimal) -> Decimal:
    """Pure function for deterministic financial rounding (Banker's Rounding)."""
    raw_settled = source_amount * fx_rate
    return raw_settled.quantize(FOUR_PLACES, rounding=ROUND_HALF_EVEN)


def evaluate_transaction_risk(payload: PaymentSettlementRequest, settled_amount: Decimal) -> tuple[int, di
    """Deterministic heuristic + statistical threshold check for cross-border anomalies."""
```

```
    risk_score = 0
    reasons = []

    # Rule 1: High-value cross-border settlement threshold
    if settled_amount > Decimal("50000.0000"):
        risk_score += 65
        reasons.append("LARGE_SETTLEMENT_VOLUME_SPIKE")

    # Rule 2: Abnormal FX rate deviation check (e.g., CNY -> MYR outside normal [0.55, 0.75] corridor)
    if payload.source_currency == "CNY" and payload.settled_currency == "MYR":
        if not (Decimal("0.550000") <= payload.fx_rate <= Decimal("0.750000")):
            risk_score += 50
            reasons.append("FX_RATE_CORRIDOR_ANOMALY")

    return min(risk_score, 100), {"flags": reasons}


async def settle_cross_border_payment(
    session: AsyncSession,
    idempotency_key: str,
    payload: PaymentSettlementRequest,
) -> PaymentSettlementResponse:
    async with session.begin():
        # 1. Check Idempotency Key First
        existing_tx_stmt = select(Transaction).where(Transaction.idempotency_key == idempotency_key)
        existing_tx = (await session.execute(existing_tx_stmt)).scalar_one_or_none()

        if existing_tx:
            wallet_stmt = select(Wallet).where(
                Wallet.merchant_id == payload.merchant_id,
                Wallet.currency == payload.settled_currency,
            )
            wallet = (await session.execute(wallet_stmt)).scalar_one()
            return PaymentSettlementResponse(
                id=existing_tx.id,
                idempotency_key=existing_tx.idempotency_key,
                merchant_id=existing_tx.merchant_id,
                source_amount=existing_tx.source_amount,
                source_currency=existing_tx.source_currency,
                settled_amount=existing_tx.settled_amount,
                settled_currency=existing_tx.settled_currency,
                fx_rate_applied=existing_tx.fx_rate_applied,
                status=existing_tx.status,
                risk_score=existing_tx.risk_score,
                new_wallet_balance=wallet.balance,
                idempotent_replay=True,
            )

        # 2. Acquire Pessimistic Row-Level Lock on the Merchant's Wallet
        wallet_lock_stmt = (
            select(Wallet)
            .where(
                Wallet.merchant_id == payload.merchant_id,
                Wallet.currency == payload.settled_currency,
            )
            .with_for_update()  # <--- Prevents concurrent race conditions!
        )
        wallet = (await session.execute(wallet_lock_stmt)).scalar_one_or_none()

        if not wallet:
            wallet = Wallet(
                merchant_id=payload.merchant_id,
                currency=payload.settled_currency,
                balance=Decimal("0.0000"),
            )
            session.add(wallet)
            await session.flush()

        # 3. Compute FX Settlement & Run Risk Triage
        settled_amount = calculate_settled_amount(payload.source_amount, payload.fx_rate)
        risk_score, risk_meta = evaluate_transaction_risk(payload, settled_amount)
        status = "FLAGGED_HIGH_RISK" if risk_score >= 60 else "SETTLED"

        # Only credit available merchant wallet balance if not flagged as high risk
        if status == "SETTLED":
            wallet.balance = (wallet.balance + settled_amount).quantize(FOUR_PLACES, rounding=ROUND_HALF_E

        # 4. Record Immutable Ledger Entry
        new_tx = Transaction(
```

```
        idempotency_key=idempotency_key,
        merchant_id=payload.merchant_id,
        source_amount=payload.source_amount,
        source_currency=payload.source_currency,
        fx_rate_applied=payload.fx_rate,
        settled_amount=settled_amount,
        settled_currency=payload.settled_currency,
        status=status,
        risk_score=risk_score,
        risk_metadata=risk_meta,
    )
    session.add(new_tx)
    await session.flush()

    return PaymentSettlementResponse(
        id=new_tx.id,
        idempotency_key=new_tx.idempotency_key,
        merchant_id=new_tx.merchant_id,
        source_amount=new_tx.source_amount,
        source_currency=new_tx.source_currency,
        settled_amount=new_tx.settled_amount,
        settled_currency=new_tx.settled_currency,
        fx_rate_applied=new_tx.fx_rate_applied,
        status=new_tx.status,
        risk_score=new_tx.risk_score,
        new_wallet_balance=wallet.balance,
        idempotent_replay=False,
    )
```

Part 5: Your First Hands-On Exercise in Antigravity

To lock in your learning right now, open your `antflow-guard` folder in **Antigravity** and complete these three tasks yourself with the agent:

1. **Wire up the FastAPI Router (`app/api/routes.py` & `app/main.py`):**
    - **Your Prompt to Antigravity:** *"Read `.antigravity/rules.md`, `app/services/settlement.py`, and `app/schemas/payment.py`. Create the FastAPI endpoint `POST /api/v1/payments/settle` in `app/api/routes.py` that extracts `X-Idempotency-Key` from the HTTP header and calls `settle_cross_border_payment`. Also wire up `app/main.py` with a lifespan context manager that initializes the database tables. Follow Rule 5 and ask me a verification question at the end."*
2. **Engineering Verification Check (Think about this while reviewing the agent's output):**
    - Look at step #1 in `settle_cross_border_payment` above. What happens if **two identical requests with the exact same new `X-Idempotency-Key`** arrive at the exact same millisecond before either one commits?
    - *Hint:* Both will pass the `existing_tx` check (`None`), and one will fail on `session.flush()` with an `IntegrityError` due to `unique=True` on `idempotency_key`. Ask Antigravity how to handle `sqlalchemy.exc.IntegrityError` gracefully so the second concurrent request doesn't crash with a `500 Internal Server Error`!
3. **Commit Your Progress:** Once you and Antigravity solve that edge case and test the endpoint via `uvicorn app.main:app --reload`, stage and commit your work (`git commit -m "feat(ledger): implement idempotent FX settlement with row-level locking"`), then let me know and we will move straight into **Phase 4 & 5: Property-Based & Concurrency Automated Testing + The AI Debugging Drill**.
